

ArkTrans Error Analysis Report

A Complete Codebook and Diagnostic Examples for the Failure Taxonomy

1 Introduction

This document is the complete error-analysis report accompanying the failure-mechanism study (RQ3) of the ArkTrans paper. It expands the two-level, five-family error taxonomy summarized in the paper into the full codebook: for every family and subfamily, we give the definition, the aggregated volume over the analyzed corpus, and one or more *real* examples excerpted from the model outputs of our experiments, together with the exact hvm compiler diagnostics that flagged them.

Corpus and labeling procedure. The analyzed corpus consists of 29,593 verified compiler diagnostics emitted by the lightweight model (Qwen3.6-27B) across the four RQ2 configurations (direct, +Reference Image, +Compiler Iterative Repair, +Image+Repair) on both translation directions. Each diagnostic is first reduced to an *error signature*, the pair of its error code and normalized message; the corpus collapses into 267 distinct signatures built on merely 50 distinct error codes. We drew a stratified sample of 3,102 diagnostics by error code at a 95% confidence level ($\pm 5\%$ margin of error), and three ArkTS experts open-coded it into the 267 signatures consolidated into the two-level codebook in 120 person-hours. A regular-expression pattern per family then generalized the codebook to all 29,593 occurrences; a blind validation on 200 diagnostics stratified over the five families yields Cohen’s $\kappa = 0.84$.

Notation in the examples. Code excerpts are quoted verbatim from the generated `.ets` files, with the original line numbers preserved on the left. The offending lines are marked with `»»` in the left gutter. File paths are relative to the `results/` directory of the replication package. Ellipses (`...`) denote elided lines.

2 Overview of the Taxonomy

Table 1 summarizes the taxonomy over the full corpus: for each subfamily, the number of distinct signatures, the total occurrence count, and its share of the corpus.

ID	Subfamily	#Sig.	#Occ.	Share
A-1	Parse errors	18	1,652	5.6%
A-2	Duplicate declarations	7	4,538	15.3%
A-3	Misplaced imports	3	344	1.2%
A-4	Source-PL keywords	2	585	2.0%
B-1	Nonexistent properties / calls	8	5,321	18.0%
B-2	Undefined names / modules	11	4,153	14.0%
C-1	Untyped object literals	5	3,906	13.2%
C-2	<code>any/unknown</code>	2	513	1.7%
C-3	Compatibility failures	53	2,932	9.9%

ID	Subfamily	#Sig.	#Occ.	Share
D-1	Non-UI statements in build()	133	1,950	6.6%
D-2	Misapplied decorators	3	201	0.7%
D-3	Illegal component nesting	13	332	1.1%
E	Missing resources	2	2,973	10.0%
Oth.	Miscellaneous	7	193	0.7%
Total		267	29,593	100%

Table 1: Overview of the two-level taxonomy over the full corpus of 29,593 diagnostics. #Sig. = distinct error signatures; #Occ. = total occurrences.

3 Family A: Broken Structure

The output is not a well-formed ArkTS file: truncated statements, duplicate declarations, misordered imports, or source-PL keywords pasted in verbatim.

3.1 A-1 Parse errors

Description. The file fails to parse: statements are truncated, brackets do not balance, or entire non-ArkTS syntax blocks (e.g., JSX-style tags) are emitted.

Example. The model emits JSX-like markup inside an ArkTS file; the parser gives up at the first tag.

```
60| render() {
61| return (
>>62| <MaterialTheme>
63| <Scaffold
64| topBar={
65| <TopAppBar
66| title="Invoice Scanner"
67| style={{ ...
```

Location: results/rq1_zero_shot/llm_direct/qwen3.6-27b/kotlin/038.ets, L60-67 (Qwen3.6-27B, direct)

Compiler diagnostic: [10505001] Declaration or statement expected.

3.2 A-2 Duplicate declarations

Description. A name is declared that ArkTS already provides, or the same declaration is emitted twice in one file — most often as an artifact of whole-file rewriting during iteration, or of a multi-file answer pasted into a single .ets file.

Example. The model merges what was clearly a multi-file answer into one output — the Markdown code fences are still there — and declares its own Color helper class, which collides with the ArkUI built-in Color; every later use site is flagged.

```
41| ``typescript
42| // entry/src/main/ets/constants/Colors.ets
>>43| export class Color {
44| // Backgrounds
45| static readonly RF_BACKGROUND = '#FBF8F3'
46| static readonly RF_CARD_BG = '#F7F3EE'
```

Location: results/rq1_zero_shot/llm_direct/qwen3.6-27b/swift/072.ets, L41-46 (Qwen3.6-27B, direct)

Compiler diagnostic: [10505001] Line 43: Duplicate identifier 'Color'.

Variant: struct name shadows a built-in component. The generated struct reuses the name of an ArkUI built-in component or of a platform symbol.

```
297|
298| @Component
>>299| struct LocationButton {
300| @Prop title: string = '';
301| @Prop isSelected: boolean = false;
```

Location: results/rq1_zero_shot/agent_harness/dsh-deepseek-v4-pro/swift/006.ets, L297-301

Also observed in: GPT-5.5 results/rq1_zero_shot/llm_direct/gpt-5.5/swift/006.ets L37 — the same struct LocationButton for the same SwiftUI sample.

3.3 A-3 Misplaced imports

Description. import statements appear after other statements, which ArkTS forbids (rule *arkts-no-misplaced-imports*).

Example. The import at line 7 follows five const declarations.

```
1| const NAME: string = '007_StoryEditor_en';
2| const UI_TYPE: string = 'Social';
3| const STYLE_THEME: string = 'Monochrome';
4| const LANG: string = 'en';
5| const BASELINE_SIZE: string = '720x1280';
6|
>> 7| import window from '@ohos.window';
8| import common from '@ohos.app.ability.common';
```

Location: results/rq1_zero_shot/llm_direct/gpt-5.5/kotlin/010.ets, L1-8

3.4 A-4 Source-PL keywords

Description. Keywords of the source language (or of a third language in the pretraining mixture) are pasted verbatim; by far the most frequent is the Swift/Java-style **new** in constructor calls.

Example. The controller field is initialized with a Java-style **new** call, four repair rounds after the same **new** shape was first reported in the direct run of this sample.

```
84| ...
>>86| private containerController: GridItemController = new GridItemController(0);
87|
88| build() {
```

Location:

results/rq2_common_strategies/compiler_repair/qwen3.6-27b/kotlin/013/013_iter2.ets, L86 (Qwen3.6-27B, +Compiler Iterative Repair, round 2)

Compiler diagnostic: [10505001] Line 86: 'new' keyword is not allowed in ArkTS

4 Family B: Nonexistent APIs

The code invokes properties, methods, or modules that do not exist in ArkUI/ArkTS. The diagnostic only reports the absence; it never names the correct alternative.

4.1 B-1 Nonexistent properties and calls

Description. A property or method is invoked on a real type, but the member does not exist — often a SwiftUI constant carried over verbatim, or a method chained on a void-returning call.

Example 1. `FontWeight.SemiBold` is a SwiftUI-flavored constant; ArkTS spells it `FontWeight.Medium`. This exact signature alone accounts for 424 diagnostics in the corpus.

```
149| .fontColor(AppTokens.Colors.primary)
150| .fontSize(18)
>>151| .fontWeight(FontWeight.SemiBold)
152| .margin({ top: AppTokens.Spacing.md })
```

Location: results/rq1_zero_shot/llm_direct/qwen3.6-27b/kotlin/001.ets, L149-152 (Qwen3.6-27B, direct)

Compiler diagnostic: [10505001] Line 151: Property 'SemiBold' does not exist on type 'typeof FontWeight'.

Example 2. The SwiftUI method-chain habit: a modifier is chained onto a statement that returns void.

```
158| // Bottom Navigation
159| this.bottomNavigation()
>>160| .margin({ left: HealthDashboardTokens.Spacing.md, right: HealthDashboardTokens.Spacing.
    md, bottom: HealthDashboardTokens.Spacing.md })
161| }
162| .width('100%')
```

Location: results/rq1_zero_shot/llm_direct/qwen3.6-27b/swift/008.ets, L158-162 (Qwen3.6-27B, direct)

Compiler diagnostic: [10505001] Line 160: Property 'margin' does not exist on type 'void'.

4.2 B-2 Undefined names and modules

Description. The code references a name or module that resolves to nothing at all — most often a SwiftUI symbol carried over verbatim. The compiler even suggests an ArkTS-shaped alternative that is itself wrong for this use: `Scroller` is a controller type, not a scrollable component.

Example. The SwiftUI container `ScrollView` is written as if it were an ArkUI component.

```
58| Stack({ alignContent: Alignment.Bottom }) {
59|   Column() {
>>60|     ScrollView() {
61|       Column() {
62|         Blank().height(24)
```

Location: results/rq1_zero_shot/llm_direct/qwen3.6-27b/swift/082.ets, L58-62 (Qwen3.6-27B, direct)

Compiler diagnostic: [10505001] Line 60: Cannot find name 'ScrollView'. Did you mean 'Scroller'?

Also observed in: Qwen3.6-27B

results/rq1_zero_shot/llm_direct/qwen3.6-27b/swift/094.ets L17 — `ScrollView({ indicator: IndicatorStyle.None })` { in the direct run of another sample.

5 Family C: Wrong Usage

The API is real, but the way it is used is forbidden by ArkTS.

5.1 C-1 Untyped object literals

Description. Object literals are passed where ArkTS demands an explicitly declared class or interface (rule *arkts-no-untyped-obj-literals*).

Example. The design-token table is copied from the Kotlin source as one deeply nested untyped literal; the diagnostic fires at the outermost brace and again at every nested level.

```
1| import { window } from '@kit.ArkUI'
2|
3| // --- Design Tokens & Constants ---
4|
>> 5| const AppTokens = {
6|   Colors: {
7|     primary: '#6750A4',
8|     onPrimary: '#FFFFFF',
```

Location: results/rq1_zero_shot/llm_direct/qwen3.6-27b/kotlin/001.ets, L1-8 (Qwen3.6-27B, direct)

Compiler diagnostic: [10605038] Line 5: Object literal must correspond to some explicitly declared class or interface (arkts-no-untyped-obj-literals)

5.2 C-2 any/unknown types

Description. ArkTS requires explicit types where TypeScript tolerates *any/unknown* (rule *arkts-no-any-unknown*).

Example 1. A TypeScript double assertion survives into ArkTS.

```
72| aboutToAppear() {
73|   // Hide navigation bar for immersive experience similar to Android code
>>74| let windowClass: window.Window = null as unknown as window.Window;
75| try {
76|   const w = window.getWindowSync(this.context);
```

Location: results/rq1_zero_shot/llm_direct/qwen3.6-27b/kotlin/007.ets, L72-76 (Qwen3.6-27B, direct)

Example 2. The same escape hatch in its single-cast form: a platform object coerced with *as any*.

```
264| // Hide navigation bar for full screen effect similar to Android code
265| try {
>>266| const w = window.getWindowSync(this.getUIContext().windowStage as any);
267| w.setWindowLayoutFullScreen(true);
268| } catch (e) {
```

Location: results/rq1_zero_shot/llm_direct/qwen3.6-27b/kotlin/001.ets, L264-268 (Qwen3.6-27B, direct)

Compiler diagnostic: [10605008] Line 266: Use explicit types instead of "any", "unknown" (arkts-no-any-unknown)

5.3 C-3 Compatibility failures

Description. The API is real but the usage violates ArkTS compatibility rules: spread operators outside arrays, comma operators outside loops, standalone *this*, limited type inference, and similar restrictions (the *arkts-** rule family). This is the most signature-diverse subfamily (53 signatures).

Example. The spread operator copies a shared design-token object into an options literal, a TypeScript idiom ArkTS forbids outside arrays.

```

118| Blank()
119| Button("New")
>>120| .font({ ...AppTokens.Typography.label })
121| .fontColor(AppTokens.Colors.onPrimary)
122| .backgroundColor(AppTokens.Colors.primary)

```

Location: results/rq1_zero_shot/llm_direct/qwen3.6-27b/swift/073.ets, L118-122 (Qwen3.6-27B, direct)

Compiler diagnostic: [10605099] Line 120: It is possible to spread only arrays or classes derived from arrays into the rest parameter or array literals (arkts-no-spread)

Also observed in: Three more in the same file: L145, L152, L155.

6 Family D: Misplaced UI Code

Structurally valid ArkTS, but code sits where UI-component syntax is required (or decorators land on ineligible variables).

6.1 D-1 Non-UI statements inside build()

Description. Plain statements (declarations, `if/const`, plain function invocations) appear inside the UI builder where only component syntax is allowed. This subfamily has the largest signature count (133) because hvigor reports it once per offending expression with the expression text embedded in the message.

Example 1. A custom component invoked as a plain statement inside the builder.

```

167|
168| // Bottom Bar
>>169| this.BottomBar()
170| }
171| .width('100%')

```

Location: results/rq1_zero_shot/llm_direct/qwen3.6-27b/kotlin/003.ets, L167-171 (Qwen3.6-27B, direct)

Compiler diagnostic: [10905204] Line 169: 'this.BottomBar();' does not meet UI component syntax.

Example 2. Ordinary statements interleaved with components in the builder.

```

186| Column({ space: AppTokens.Spacing.sm }) {
187|   ForEach(this.questions[this.index].options, (option: string, i: number) => {
>>188|     const isActive = this.selected === i;
189|
190|     Button(option) {

```

Location: results/rq1_zero_shot/llm_direct/qwen3.6-27b/kotlin/005.ets, L186-190 (Qwen3.6-27B, direct)

Compiler diagnostic: [10905209] Line 188: Only UI component syntax can be written here.

6.2 D-2 Misapplied decorators

Description. State decorators (`@Prop`, `@ObjectLink`, ...) are applied to variables of a type the decorator does not support — most uniformly, to function-typed callback properties.

Example.

```

173| struct IsoSelector {
174|   @Prop currentIso: string
>>175|   @Prop onIsoSelected: (iso: string) => void
176|
177|   build() {

```

Location: results/rq1_zero_shot/llm_direct/qwen3.6-27b/kotlin/090.ets, L173-177 (Qwen3.6-27B, direct)

Compiler diagnostic: [10905363] Line 175: The V1 decorator '@Prop' cannot be applied to a Function-type variable 'onIsoSelected'.

Also observed in: Two more in the same file (L212, L263); Claude Code results/rq1_zero_shot/agent_harness/claude-code/swift/093.ets L231 (@Prop onClick: () => void); DeepSeek-V4-Pro results/rq1_zero_shot/llm_direct/deepseek-v4-pro/swift/097/Index.ets L240 (@Prop action: () => void). The same shape across model families.

6.3 D-3 Illegal component nesting

Description. Children are placed under components that cannot host them: a labeled `Button` with children, multiple roots in `build()`, or components outside their required parent.

Example. `Button('label') { child }` is illegal — either a label or a child, not both. Here the child even repeats the label text.

```

331| Blank()
332|
>>333| Button('Mark All Read') {
334|   Text('Mark All Read')
335|   .fontSize(14)

```

Location: results/rq1_zero_shot/llm_direct/qwen3.6-27b/kotlin/003.ets, L331-335 (Qwen3.6-27B, direct)

Compiler diagnostic: [10905202] Line 333: The Button component with a label parameter can not have any child.

Also observed in: Qwen3.6-27B results/rq1_zero_shot/llm_direct/qwen3.6-27b/kotlin/005.ets L140 — `Button("Restart") { ... }`.

7 Family E: Missing Resources

The code references colors, strings, or media that do not exist in the target project. The syntax of the reference is standard; the referenced file simply is not there.

Example 1. The model needs a weather icon, knows the target project has none, and invents a plausible name — its own comment says *placeholder*; the same invented names are reused throughout the file.

```

49| // Heat Mode Icon Button (Top Right)
50| Row() {
>>51|   Image($r('app.media.ic_sunny')) // Placeholder for WbSunny icon
52|   .width(24)
53|   .height(24)

```

Location: results/rq1_zero_shot/llm_direct/qwen3.6-27b/kotlin/062.ets, L49-53 (Qwen3.6-27B, direct)

Compiler diagnostic: [10903329] Line 51: Unknown resource name 'ic_sunny'.

Also observed in: The same file reuses the invented names at L128–129 (ic_wind, ic_sunny).

Example 2. The model’s own comment concedes it is guessing a resource that may not exist.

```
278| // Top Bar
279| Row() {
>>280| Image($r('app.media.ic_back')) // Placeholder for back icon, using system default if
    |     available or custom
281|     .width(24)
282|     .height(24)
```

Location: results/rq1_zero_shot/llm_direct/qwen3.6-27b/kotlin/011.ets, L278–282 (Qwen3.6-27B, direct)

Compiler diagnostic: [10903329] Line 280: Unknown resource name 'ic_back'.

8 Miscellaneous (Oth.)

This category collects the residual diagnostics that the coders could not consistently assign to the five families (193 occurrences, 0.7% of the corpus). Following the convention of keeping the codebook complete but focused, we do not present examples for this residual category.